into a set of k indices, pointing to the bits that will hold the information about a key (stored vs not stored).

The set of functions will be created by using double hashing to combine the two arguments in k different way. With respect to linear or quadratic hashing, double hashing will increase the number of possible hashing functions that we can obtain, from $O(k)$ to $O(k^2)$. Still, this is far from the ideal $O(k!)$ guaranteed by uniform hashing, but in practice it is close enough to hold good performance (meaning: a low collisions rate).

Listing 4.8 Method initHashFunctions

```
function initHashFunctions(numHashFunctions,
numBits)                          #1
    return range(0, numHashFunctions).map(i => (h1, h2) =>
          floor(h1 + i * h2 + i * i) mod
numBits)                              #2
```

#1 Method `initHashFunctions` takes the number of desired functions and the number of bits held by the Bloom filter, and creates and return a list of *double hashing* functions, taking two values and returning their hash.

#2 We use again functional notation, applying a lambda function to an array: for convenience, we map from integers from `0` to `numHashes-1` (included) into a list of equally many *double hashing* functions.

4.6.5 Constructor